

Juju-chu じゅじゅちゅ!



大岡由佳



リnewで始める

Jujutsu×AIワークフロー

初めてのJujutsuは、この1冊から!

人気急上昇中の新世代VCS

領域展開

JujutsuでAIエージェントを安全に使いこなす

じゅじゅちゅ！

jj new で始める Jujutsu × AI ワークフロー

大岡 由佳

くるみ割り書房

※本書に記載されている会社名、製品名などは一般に各社の商標、または登録商標です。

※本書に記載された URL、バージョンなどは予告なく変更される場合があります。

※本書の内容については最大限の努力をもって正確を期していますが、これに基づき生じた運用結果には責任を負いかねますので、ご了承ください。

まえがき

Jujutsu が今、注目を集めています。

バージョン管理システム (VCS) といえば長らく Git 一強の時代が続いており、特に 2010 年代以降に業界に入ったソフトウェアエンジニアの中には、Git しか使ったことがない人も多いでしょう。また GitHub は Git をデファクトスタンダードの地位に押し上げた最大の立役者であり、そのアカウント数は全世界でなんと約 2 億。もはや「GitHub アカウントを持たないエンジニアはエンジニアにあらず」とも言われかねない普及ぶりです。

そんな風潮の中、Git 以外の VCS が話題になるのは非常にめずらしく、「事件」と言ってもいい出来事ではないでしょうか。

なぜ今、Jujutsu なのか？

Jujutsu は 2019 年から開発が始まりましたが、最初の公開リリース (v0.3.0) が出たのは 2022 年のこと。しかし開発コミュニティでしばしば話題に上がるようになったのは、2025 年に入ってからです。折しも AI コーディングエージェントが急速に普及し、「パイプコーディング」なる言葉がパスワードとして広まり出した時期と一致します。

AI エージェントが人間とは比較にならない速度で大量のコードを生成・変更してくれるようになり、コードを書くコストや試行を繰り返すコストが大幅に下がりました。そんな状況と、「人間が 1 行 1 行丁寧にコードを書き、何を記録するかを自分で選び、準備が整ってから慎重に共有する」——そういうワークフローを前提に設計された従来の VCS の間に、齟齬を感じる人が増えてきたのではないのでしょうか。

そして気が付くとそこに Jujutsu という新しい VCS があった。試してみたら AI エージェントによる開発のスピード感到すごくマッチする。この魅力を他の人たちにも伝えねば！ と、昨今の急激な盛り上がりはこのようにして生まれていると筆者は見ています。

「でもそれ、Git でもできるよ」

紹介記事などがバズって Jujutsu が話題になるたびに必ずと言っていいほど見かける反論(?)に、「同じことは Git でもできる」というものがあります。しかし Jujutsu の価値は、「Git にできないことをやる」ことにはありません。

たとえばガラケーと iPhone。両者とも同じく電話ができて、Web ページが見られて、アプリをインストールして使えて、カメラで写真が撮れます。そして iPhone の登場当初には、実際にそのように指摘して iPhone を否定する人たちも少なからずいました。しかし最終的な両者の評価がどうなったかは周知のとおり。できることはたとえ同じであっても、そこで提供されるユーザー体験には決定的な差があったのです。

「できる」と「快適にできる」はちがいます。Jujutsu では Git と同じことをやるにあたっても、それを自動でやってくれたり、操作がおぼえやすかったり、失敗しても簡単にやり直せたりなど、より安心・安全・手軽になっています。

キーワードは「認知負荷の低さ」と「心理的安全性」。Jujutsu はすべてにおいて、この 2 つが手厚く配慮されているのです。

本書の対象読者

本書が対象として想定する読者像は、以下のようなものです。

- Git の基本的な操作を知っており、日常的に Git および GitHub / GitLab を利用している
- Claude Code や Codex CLI のような AI コーディングエージェントを使っている、もしくは始める予定がある
- Jujutsu は未経験、もしくは使い始めたがまだ初心者の段階

「Git 自体がまだよくわからない人向けの入門書」ではないので、お気をつけください。なお VCS 実演のための教材として React を使っていますが、それが本題ではないため React および Node.js エコシステムについての知識は必須ではありません。適宜、ご自身の主戦場の環境に読み替えてお進みください。

本書を読む上での注意事項

Jujutsu は 2026 年 4 月現在、非常に活発に開発されており、まだ安定版 (v1.0.0) には達していません。実際、プロジェクトの README でも自身を「experimental version control system」と位置づけており、v1.0.0 までは破壊的変更が入る可能性がある」と明言しています。したがって、本書の記述と最新版の間で各種名称や挙動に差異が生じることがあります。もっとも、本書の内容は単にコマンドの使い方にとどまらず、Jujutsu の設計思想やメンタルモデル、そして AI 支援開発への応用などに及ぶ包括的なものです。今後、表面的な変更があっても、本書の内容は将来にわたって有用だと考えています。

本書の構成

第 1 章では Jujutsu の概要、第 2 章では基本操作、第 3 章で AI との協調開発、第 4 章で各種の応用、第 5 章で FAQ とトラブルシューティングを扱っています。最初から順に読むことをおすすめしますが、まず動かしてみたい人は第 2 章から、AI 連携に強い関心がある人は第 3 章を急いで読んでもよいでしょう。また第 4 章と第 5 章は辞書的にも利用できるようになっています。

なお本書は、シニアエンジニアとジュニアエンジニアの 2 人の対話によるストーリー形式で展開されます。気軽に読んでいただけるよう、また初心者が抱きがちな疑問に逐一答えられるようにこの形式を採用しています。

それでは、これから Jujutsu の世界へご案内いたします。開始の呪文は `jj new`。

本書について

登場人物の紹介

柴崎雪菜（しばさき ゆきな）

都内のとあるインターネットサービスを運営する会社のシニアエンジニア。Reactの腕を買われて現在の会社にジョインし、テックリードとしてフロントエンド開発チームを一から育てた。新しい技術を見つけてきては見切り発車で導入しようとする傾向があり、チームメンバーからは多少辟易されているが、それらは不思議と後からブレイクすることが多いため黙認されている。

秋谷香苗（あきや かなえ）

柴崎と同じチームに所属する、やる気あふれるジュニアエンジニア。柴崎の一番弟子を自任しており、彼女のやることは何でも真似しようとする。アニメや漫画が大好き。最近では「AI に仕事を奪われたらどうしよう」と不安を抱きつつも、積極的に AI エージェントを開発に利用して、急激な業界の変化に置いていかれないよう努力している。

サンプルコードについて

本文で紹介している各種設定のサンプルコードは、GitHub に用意した下記のリポジトリで公開しています。それぞれのファイルの場所は、対応する本文の脚注をご参照ください。

- ・『じゅじゅちゅ！ jj new で始める Jujutsu x AI ワークフロー』サポート用公開リポジトリ
<https://github.com/klemiuary/Juju-chu-ja>

本文および上記リポジトリに掲載しているコードは、読者の判断でご自由にお使いください。公開文書や動画などに引用するにあたっては、出典を明記していただければ筆者に許可を求める必要はありません。なおこれらのコードをそのまま転載することはご遠慮ください。

正誤表

本文内の記述内容の誤りや誤植についての正誤表は、以下のページに掲載しています。随時更新の予定です。

- 『じゅじゅちゅ！ jj new で始める Jujutsu × AI ワークフロー』正誤表
<https://github.com/klemiuary/Juju-chu-ja/blob/main/errata.md>

本書内で使用している主なソフトウェアのバージョン

- Jujutsu (jj) 0.40.0
- jjui (jjui) 0.10.2
- JJ View (jj-view.jj-view) 1.22.0
- Claude Code (@anthropic-ai/claude-code) 2.1.92
- Codex CLI (@openai/codex) 0.118.0

目次

まえがき	3
本書について	6
登場人物の紹介	6
サンプルコードについて	6
正誤表	7
本書内で使用している主なソフトウェアのバージョン	7
プロローグ	13
第1章 Jujutsu ってどんなツール？	16
1-1. Git は AI 支援開発と相性が悪い？	16
冗長な 3 ステートモデル	16
コンテキスト切り替えのコストの高さ	18
履歴の書き換えが複雑で危険	18
コマンドの副作用が大きく取り消しにくい	19
1-2. AI 時代に評価される Jujutsu の特徴	21
作業ディレクトリが即 commit される	24
あらゆる操作が undo 可能	25
Conflict をただの状態として扱う	26
第2章 使ってみよう Jujutsu	29
2-1. Jujutsu の環境構築	29
2-1-1. Jujutsu のインストール	29
2-1-2. Jujutsu の初期設定	30
2-2. Jujutsu 体験ツアー	32
2-2-1. リポジトリの初期化	32
2-2-2. リポジトリの状態を確認する	34
2-2-3. Change の操作	37
2-2-4. リモートとやりとりする	42
2-3. Jujutsu の 3 つのログ	45
2-3-1. Revision Log (jj log)	45

2-3-2. Evolution Log (<code>jj evolog</code>)	48
2-3-3. Operation Log (<code>jj operation log</code>)	54
2-4. Jujutsu のメンタルモデル	57
2-4-1. Change とは何か	57
2-4-2. Branch と Bookmark のちがひ	58
2-4-3. 匿名 Branch で作業する	59
2-5. よく使う <code>jj</code> コマンド一覧	61
状態確認	61
revision 操作	61
履歴の整理	62
操作の取り消しと追跡	62
bookmark およびリモートとの連携	62
リカバリ	62
第3章 実践 Jujutsu × AI ワークフロー	63
3-1. AI エージェントを Jujutsu と協調させる	63
3-1-1. AI エージェントに Jujutsu を使わせる	63
3-1-2. <code>jj</code> コマンドの Permissions 設定	69
3-1-3. Hooks で <code>jj fix</code> を走らせる	72
3-2. 実例で見る Jujutsu × AI による開発プロセス	77
3-2-1. AI が作成した change の粒度を整える	77
3-2-2. push して PR を作成する	83
3-2-3. workspace を使った並行開発	88
第4章 Jujutsu の高度な術式	96
4-1. ターゲットを指定する冴えたやりかた	96
4-1-1. Revset で Revision をスマートに指定する	96
シンボル	96
演算子	97
関数	98
文字列パターン	99
4-1-2. Fileset でファイルをスマートに指定する	99
ファイルパターン	100
演算子	100

4-2. 知ってると便利な裏技的コマンド	101
4-2-1. <code>jj absorb</code>	101
4-2-2. <code>jj arrange</code>	102
4-2-3. <code>jj bookmark advance</code>	103
4-3. Git Hooks の代替戦術	105
4-4. Jujutsu の UI ツール	109
4-4-1. <code>jjui</code>	109
4-4-2. <code>JJ View</code>	113
第 5 章 Jujutsu 問題解決ガイド	116
5-1. FAQ	116
5-1-1. Git との比較	116
❷ Git にできて Jujutsu にできないことは？	116
❷ merge コマンドはないの？	117
❷ pull コマンドはないの？	118
❷ Git の cherry-pick 相当のことがしたい	119
5-1-2. ニッチな操作・設定について	119
❷ @ を移動させずに、任意の時点でのファイルの内容を確認できる？	119
❷ change の分割を時系列で行いたい	120
❷ 一時的なログやダンプなど、履歴に含めたくないファイルがある	123
❷ Jujutsu のリポジトリ用設定を、そのリポジトリ自身に登録したい	124
❷ track 中の bookmark の名前を変更したい	125
5-1-3. Jujutsu のトリビア	126
❷ Jujutsu は Git のラッパーツールなの？	126
❷ Jujutsu の作者ってどんな人？	128
❷ プロダクト名の意味は「呪術」「柔術」どっち？	129
5-2. トラブルシューティング	130
Ⓢ ただの push が知らないうちに force push になっている	130
Ⓢ 『Error: The working copy is stale』という謎のエラーが出る	132
Ⓢ いつのまにか change に divergent という注釈がついていた	133
Ⓢ 画像や動画ファイルを Jujutsu が追跡してくれない	135
Ⓢ PR をマージ後に fetch したら @ が迷子になる	137
Ⓢ まだ作業中のリモートの bookmark を GitHub 上から削除してしまった	138

☹ Claude Code の Permissions 設定で <code>jj log</code> を <code>allow</code> にしていても実行許可を求められる	139
エピローグ	140
著者紹介	142

第 1 章 Jujutsu ってどんなツール？

1-1. Git は AI 支援開発と相性が悪い？

「さっき秋谷さんがやらかしたような事故だけど、あれは秋谷さんが特別に不注意だったから起きたわけじゃない。VCS として使ってる Git の設計が最新の AI 支援開発にフィットしていないために、起こるべくして起こったものとも言えるだろうね。まあ 20 年以上前に設計されたツールなので、AI エージェントがこんなに大量のコードを次々に生成してしまうような事態を想定しろというのは無理な話だけど」

「.....そうか、そうだったんですね。私は悪くない！」

「Jujutsu の説明に入る前に、Git のどこが AI 支援開発と相性が悪いかというのを確認しておこうか。そのほうが Jujutsu を学びながらその便利さ、使いやすさを実感できると思うから」

冗長な 3 ステートモデル

「Git の最大の特徴であり、同時に初学者が最初につまずきやすいところでもあるんだけど、Git ではファイルの変更が履歴に記録されるまでに次の 3 つの状態を経由するようになってる」

- **Working Tree** (作業ツリー) ファイルを実際に編集する場所。working directory ともいう
- **Staging Area** (ステージング領域) working tree と repository の間にある、commit の準備領域。index ともいう
- **Repository** (リポジトリ) 変更履歴やすべてのファイルが保存されているデータベース

1-1. Git は AI 支援開発と相性が悪い？

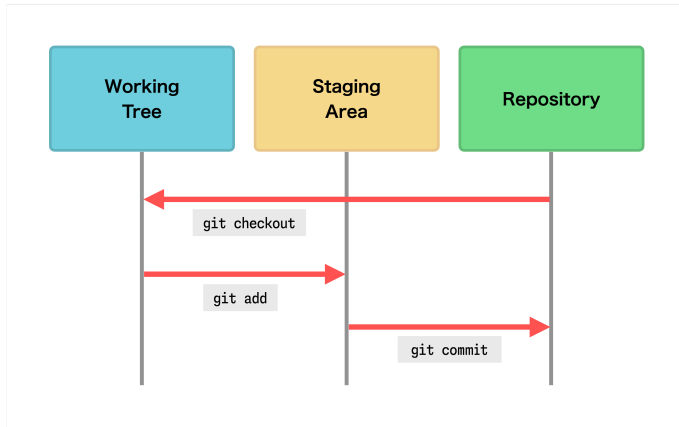


図 1: Git の 3 ステートモデル

「そうそう。最初、なんでわざわざ `git add` して `git commit` する必要があるんだろうと思いましたね。commit だけでいいじゃないですか」

「人間が手作業で開発していた時代には、このモデルは一定の合理性があったんだよ。working tree から変更の一部だけを選んで index して、そこから意味のまとまりごとに commit するという手順を想定してる。staging area は次の commit の下書きに使うためのものなの。これがあることで慎重に作業できて、commit の品質が上げやすかったのね。ほら、Git はもともと Linux カーネルの開発のために作られたものだったから、汚くて意図が読みづらい commit を上げられるとメンテナが迷惑するでしょ」

「ふーむ、なるほど」

「でも AI エージェントが複数ファイルにまたがる数十～数百行のコードを一気に生成してしまう現代においては、このステップが大きな足枷になってる。疲れ知らずの AI が次々に書きあげてくるコードを、人間がいちいち `git add` と `git commit` を実行し続けるのは現実的じゃない。その結果、履歴に保存しないまま AI に次の指示を出してしまい、手戻りできなかったり履歴の海で溺れるという事故が頻発するというわけ」

コンテキスト切り替えのコストの高さ

「実際の開発では、不意に思いついたアイデアを試したくなったり、急なレビュー依頼やバグ修正のタスクが発生することがあるよね。すると working tree を中途半端な状態のまま別の branch に切り替えることを余儀なくされるわけだけど、そういうとき Git では一時退避のために `git stash` を使う」

「はいはい、stash は毎日のようにやってますね。あれ面倒だし、戻ってきたときに stash してたことを忘れてたりすると、悲惨なことになるんですよ」

「割り込みタスクの作業が長引いている最中に、さらに緊急度の高い割り込みタスクが起きたりして stash を複数積み上げたりすると、どれがどの作業の退避データだったかわからなくなるよね。それで復元時に conflict を起こして泣く泣く変更を破棄する、なんて事態もめずらしくない」

「うわ、想像しただけで胃が痛くなりますね……」

「それから Git での作業で、馬鹿にならないのが名前づけのコスト。新しく作業を始めるにはまず branch を作り、それに適切な名前を付ける必要がある。commit するにもまず名前が必要で、そして一度決めた名前を思い直して後から変更するのは、できなくはないけど煩雑な手続きが必要になる。ちょっと実験してダメならすぐ破棄するようなものであっても、作業の前に何かしら名前を考えなければいけないのは、認知的な負荷が高い」

「名前づけのコストですか……。あまり意識してなかったですけど、言われてみれば毎回、どんなのつけたらいいか迷ってますね」

履歴の書き換えが複雑で危険

「人間が慎重にコードを書いていたときと比べて、AI にコードを書かせるコストは劇的に低いので、いったん履歴に保存したものを後から変更したくなる事態は格段に増えてる。ところが Git で履歴を書き換える操作はハードルが高いのよね」

「わかります。私もどうしても必要になったときは検索したりチャット AI に尋ねたりしながら、恐る恐るやってますもん」

「Git では履歴の書き換えというひとつの概念に対して `git commit --amend`、`git rebase -i`、`git reset` といった複数のコマンドがあって、さらにオプション

の与え方によって作用範囲がガラッと変わったりするのでメンタルモデルの構築が難しい。これらをすべて暗記してて、場面によって適切に使いこなせる人は少ないと思う」

「ですよねー」

「特に `git rebase -i` はエディタ上で commit 一覧を編集するという独特の UI で、ここでの操作ミスが直接履歴に影響してくる。行を消せばその commit が消えるし、`pick` を `drop` に書き換えても消える。エディタを保存して閉じた瞬間に処理が走り始めるので、編集をまちがえたときの絶望感は半端ない。

また途中で conflict が発生すると、commit をひとつずつ順番に適用していく中で何度も conflict の解消を求められることもよくある。その果てに今どの commit を処理中なのか見失って、けっきょく `git rebase --abort` で全部やり直すか、混乱したまま `git rebase --continue` を連発するかの二択なんてことになりがち」
「はい。私はそれが怖いので、後から分割・修正しなくて済むように、未 commit の状態で行けるだけ行きたいな感じになっちゃうんですよね。それでさっきみたいな事故が起きたり、泥だんごのような commit で PR を出しちゃったりすることになるんですけど」

コマンドの副作用が大きくなり取り消しにくい

「Git には、一度実行すると簡単には取り返しがつかない破壊的なコマンドが少なくない。たとえば `git reset --hard`、`git clean -fd`、`git restore` なんかはまだ commit してない変更を容赦なく吹き飛ばす。苦勞して working tree に積み重ねた変更が、たったひとつのコマンドの打ちまちがいでデータ消失につながるのは恐怖以外の何ものでもない」

「以前 Gemini が私の意図とちがう変更をしてくれたので『元に戻して』と言ったら、`git restore` を実行されてそれ以前の作業も全部消されてしまったことがあります。『大変申し訳ありませんでした』って謝られたんですが、後の祭りですよ」

「Claude Code なら permissions 設定で特定の Git コマンドの実行を禁止できるけど、設定をうっかり間違えることもあるしね。積み上げた変更を一発で不可逆的

第1章 Jujutsu ってどんなツール？

に壊すことができる Git を AI にさわらせるリスクは高いよ。

また Git のヘビーユーザーには『git reflog を使えば操作を戻せるよ』みたい
に言う人がいるけど、reflog は何でも元通りに戻せるわけじゃない。それに reflog
の解説は難解だし、安全に元の状態に復元するには Git の内部構造に対する高度
な知識が求められる。パニックになっている状態で冷静に reflog を操作して、意
図通りの状態に戻せるエンジニアはこれまたごく少数だと思う」

まとめ：Git の設計思想と AI 時代におけるミスマッチ

「うーむ、聞いてると Git はまさに人類には早すぎるツールに思えてきました。ど
うしてこんな仕様になってるんでしょうか？」

「それは Git が元々、『人間が手作業でコードを書き、その変更を整理し、レビュー
可能な単位にまとめて共有する』という Linux カーネルの開発モデルを念頭に設
計されてるからだね。だから変更を人間同士で慎重に受け渡すワークフロー
においては非常に優れたツールで、長年にわたって存分にその実力を発揮してき
た。ただ AI 支援開発のシーンでは事情がかなり違ってくる」

- 高速な試行錯誤が起きる
- 大量のコード変更が一気に発生する
- 作業のコンテキストが頻繁に切り替わる
- 後から履歴を整理したい場面が増える

「こういった条件の元では、これまでワークしてきた Git の特徴が開発の効率を
落とす原因になってしまう。それが AI 時代に Git の使いづらさが際立ってきた
理由だね」

「ふむふむ」

「AI 支援開発で求められるのは、まず雑に試してその後で安全に整えられる ツー
ルなんだよね。そしてまさにこの部分に対して、別の設計思想を持ったバージョ
ン管理システムが注目されるようになってきた」

「なるほど、それが Jujutsu なんですね！」

1-2. AI 時代に評価される Jujutsu の特徴

「雪菜さんを信じないわけじゃないですけど、Jujutsu の人気が急上昇中というのは本当ですか？ これまで私はそんな VCS があるって聞いたことがなかったの
で」

「まあ 2026 年前半の今はまだ、アンテナが高い一部の開発者が熱狂的に支持して
るって段階かな。キャズム理論⁴で言えばアーリーアダプタに広まっている段階
で、まだキャズムを超えられてない。

何か資料を示してあげたいんだけど、VCS の人気を客観的に計る指標という
のはなかなかなくてね。Git 一強時代が長かったので、開発者を対象にしたアン
ケートでもわざわざ『どんな VCS を使ってますか？』という質問が用意される
こともない。あまりフェアとは言えないんだけど、GitHub 公式リポジトリのス
ター数の遷移を見てみようか」

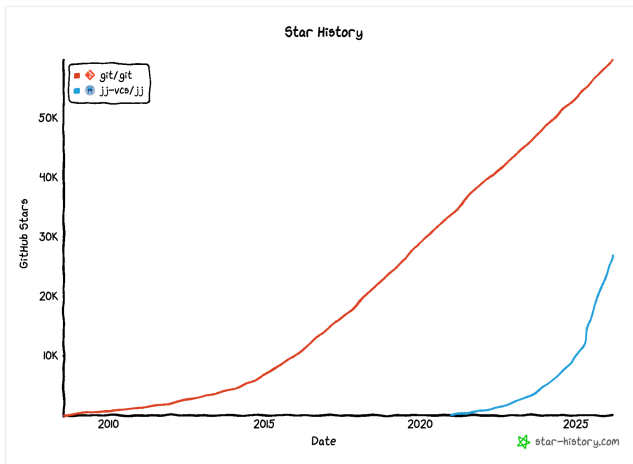


図 2: Git と Jujutsu の GitHub スター数推移 (GitHub Star History, Mar 2026)

⁴新製品やイノベーションの受け入れを説明する社会学的モデル (テクノロジー採用ライフサイ
クル) にもとづくマーケティング理論。新製品が普及する過程を 5 つの顧客層に分け、特に初
期市場とメインストリーム市場の間の「深い溝 (キャズム)」をいかにして越えるかが最重要の
課題とされる。

「おおっ！ Jujutsu のグラフ、ものすごい急カーブの放物線を描いてますね。特に 2025 年中盤からどんとブーストがかかってる感じ。このまま行けば、Git を抜く日も遠くなさそうじゃないですか。でも『フェアじゃない』っていうのは？」

「Git は GitHub 上で開発しているわけじゃなくて、ミラーリポジトリを置いてるだけなんだよね。公式の本体は Linux カーネルと同じく kernel.org でホスティングされてるの⁵。いっぽう Jujutsu は GitHub リポジトリが公式となっている」

「でもまあ、一般ユーザーにとっては GitHub がすべてみたいなものですから、人気のバロメーターとしては信頼できるんじゃないですか。Jujutsu が急激に人気を集めているというのはわかりました。では、この人気の要因は何なのでしょう？」

「秋谷さん、さっき 2025 年中盤からブーストがかかってるって言ったよね。その時期に何があったかおぼえてる？」

「うーん、何でしたっけ？」

「5 月に Claude Code が正式にリリースされたよね。それまでのコーディング AI ツールは IDE の形で提供される、サジェストとチャットによる簡単なコード変更が主体のものだった。Claude Code はターミナル上で動作するエージェント型コーディング AI ツールで、指示を与えれば自律的に高品質なコードを一気に生成してくれる。Claude Code の登場によって、人間が書くコードより AI が書くコードのほうが圧倒的に多いという状況が生まれたの」

「そうでしたね。もう遠い昔のように思えますけど、つい最近まですべてのコードを自分で手書きしてたんですよ」

「Jujutsu の人気は、Claude Code や Codex のような AI コーディングエージェントの普及に引られる形で伸びてきているように見える。私もこの時期に立て続けに書かれた AI 支援開発と絡めて紹介する記事を読んだことで、Jujutsu を始めてみようと思ったの。反響が大きかった記事にはこんなものがある」

⁵ <https://git.kernel.org/pub/scm/git/git.git>

⁶ <https://slavakurilyak.com/posts/use-jujutsu-not-git>

第 2 章 使ってみよう Jujutsu

2-1. Jujutsu の環境構築

2-1-1. Jujutsu のインストール

「ここからは Jujutsu を実際に使いながら学んでいってもらいたいんだけど、何はともあれまずはインストールだね。Jujutsu は Rust で開発されていることもあって、公式ドキュメントでは Cargo による手順が優先的に書かれてるんだけど、Rust 大好きユーザーでもない限りこれは避けたほうがいい。ビルドに時間がかかるし、アップデートの管理が煩雑になるからね」

「はい。当面 Rust をさわる予定はないので、このためだけに Rust 環境の構築から始めるのは遠慮したいです」

「幸い私も秋谷さんも Mac を使ってる。だから Homebrew を使えば以下のように簡単にインストールできるよ」

```
$ brew install jj
```

「それからこれは必須じゃないんだけど、シェルに設定を書いておくとコマンド実行時に便利だよ。自分が使ってるシェルに応じて、設定ファイルに次の 1 行を追記しておこう」

```
# zsh の場合
source <(COMPLETE=zsh jj)

# fish の場合
COMPLETE=fish jj | source
```

「これは何をやってるんですか？」

「`COMPLETE=zsh` と環境変数をセットした状態で `jj` を実行すると、zsh 用のシェ

ル補完スクリプトが出力されるの。zsh 起動時にこれを読み込ませることで、jj コマンドの最新の補完定義が動的に生成・適用されるようになる。サブコマンドやオプションを途中まで書いて Tab キーを押すと候補一覧を表示してくれたり、コマンド入力を補完してくれたりするの」

「へー、便利そうですね。私も設定に入れとこうっと。ところで自宅には Windows マシンもあるんですけど、Windows でのインストールはどうしたらいいんでしょうか？」

「WSL¹² 環境なら Mac と同じく Homebrew を使うのがいいだろうね。ネイティブ環境なら WinGet に `jj-vcs.jj` という名前でパッケージが登録されてるので、それをインストールする。ただし改行コードやシンボリックリンクの取り扱いに注意点があるので、公式ドキュメントの該当ページ¹³をよく読んでおいてね」

「わかりました」

「それから Jujutsu とは直接関係ないけど、GitHub CLI (`gh`)¹⁴ をもしまだ入れてなかったら、これを機にインストールしておこう。jj に加えて `gh` コマンドを AI エージェントに実行させることで、ワークフローがかなり効率的になるよ」

2-1-2. Jujutsu の初期設定

「Jujutsu には設定可能な項目が膨大にあるけど、最初に必要なものはごく一部だけ。まずはユーザー名とメールアドレスを登録しておこう」

```
$ jj config set --user user.name "kanaetti"
$ jj config set --user user.email "kanaetti@skydome.co.jp"
```

¹²Windows Subsystem for Linux のこと。Microsoft が提供する、Linux のバイナリ実行ファイルを Windows 上でネイティブ実行するための互換レイヤー。仮想マシン上で本物の Linux カーネルが動作し、Ubuntu や Debian といったディストリビューションをインストールして使うことができる。

¹³「Working on Windows - Jujutsu docs」<https://docs.jj-vcs.dev/latest/windows/>

¹⁴<https://cli.github.com/>

「`jj config set`¹⁵ は設定ファイルに設定値を書き込むコマンド。`--user` ならユーザーレベルの、`--repo` ならそのリポジトリ限定の設定がターゲットになる」

「これ、設定をしないまま使い始めるとどうなるんですか？」

「履歴の `author` 情報が欠落してしまい、そのままだとリモートに `push` できない。だからあらためてユーザー情報を設定したうえで、`jj metaedit --update-author <TARGET>`¹⁶ を実行して履歴の `author` 情報を書き換える必要がある」

「面倒ですね。新規インストール時には忘れないようにしないと」

「それからエディタの設定も変更しておこう。そのままだと必要な際に Jujutsu によって起動されるエディタが、`nano` という馴染みのないものになってしまう¹⁷」

```
# Vim の場合
$ jj config set --user ui.editor "vim"

# VS Code の場合
$ jj config set --user ui.editor "code --wait"
```

「VS Code の起動で渡してる `--wait` オプションって何ですか？」

「Jujutsu がエディタを起動する際、その終了を検知して編集内容を受けとる必要がある。GUI エディタをターミナルから開くと通常は別プロセスで起動されて、編集内容が宙に浮いてしまう。`--wait` をつけると VS Code は開いたタブが閉じられるまでそのプロセスをブロックし続けるので、Jujutsu がその完了を認識して編集内容を正しく受け取れるようになるの」

「ふむふむ、なるほど」

¹⁵ 「`jj config set` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-config-set>

¹⁶ 「`jj metaedit` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-metaedit>

¹⁷ Mac および Ubuntu 系 Linux で環境変数 `$EDITOR` が未設定時の挙動。Windows の場合はメモ帳 (Notepad) が起動する。

「ではこの状態で `jj config edit --user`¹⁸ を実行してみよう。さっき指定したエディタで次のような内容が表示されるはず」

```
#:schema https://docs.jj-vcs.dev/latest/config-schema.json

[user]
name = "kanaetti"
email = "kanaetti@skydome.co.jp"

[ui]
editor = "vim"
```

「それからこのファイルはどこにあるのか。それも `jj config` で知ることができる」

```
$ jj config path --user
/Users/kanaetti/.config/jj/config.toml
```

「このファイルを直接編集することでも、その設定が反映されるんですよね？」
「もちろん。だからこのファイルを Dropbox の共有フォルダなどに置いて、そこからシンボリックリンクを張れば、複数マシンで Jujutsu のユーザー設定を共有できたりするよ」

2-2. Jujutsu 体験ツアー

2-2-1. リポジトリの初期化

「じゃあ Jujutsu を使ってファイルのバージョン管理をやっていこう。React アプリを教材にするのがいいかな。ランタイムに Node.js、パッケージ管理に pnpm を利

¹⁸ 「`jj config edit` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-config-set>

用するので、入れてない場合は先に準備しておく。そしてこれらのインストールには `mise`¹⁹ を使う」

```
$ brew install mise
$ mise use -g node pnpm
```

「じゃあ React プロジェクトを作成して、そこを初期化。Jujutsu の管理下に置こう」

```
$ pnpm create vite jj-tour
◇ Select a framework:
| React
◇ Select a variant:
| TypeScript
◇ Install with pnpm and start now?
| No
└ Done.

$ cd jj-tour/
$ pnpm install
$ jj git init
Initialized repo in "."
Hint: Running `git clean -xdf` will remove `./jj/`!

$ ls -d .*
.git/      .jj/      .gitignore
```

「初期化のコマンドって `jj init` じゃなく `jj git init`²⁰ なんですか？」

「Jujutsu はプラグイン可能なストレージアーキテクチャを採用していて、Git をバックエンドのストレージとして使うことができる。だからこのコマンドは

¹⁹ プログラミング言語のランタイムや各種 CLI ツールを、複数バージョンを管理しつつインストールできるパッケージマネージャ。タスクランナーと環境変数の設定機能を併せ持つ。
<https://mise.jdx.dev/>

²⁰ 「`jj git init` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-git-init>

Jujutsu を有効にした Git リポジトリを作成するものね。古い記事では Jujutsu リポジトリと Git リポジトリの共存を有効にするために `--colocate` オプションが必要とされていることがあるけど、今ではデフォルトで指定されるので不要「すでに Git で管理してるリポジトリで、後から Jujutsu を使いたい場合はどうしたらいいんでしょう？」

「同じく `jj git init` で OK。Git で作成した過去の commit 履歴は、Jujutsu の履歴としてそのまま取り込まれるよ」

2-2-2. リポジトリの状態を確認する

「次は、このリポジトリの現在の状態を確認してみよう」

```
$ jj status
Working copy changes:
A .gitignore
A README.md
A eslint.config.js
A index.html
:
A vite.config.ts
Working copy (@) : uwzmrwo ba953de6 (no description set)
Parent commit (@-): zzzzzzzz 00000000 (empty) (no description set)
```

「`jj status`²¹ は `git status` とほぼ同じ目的のコマンドと考えていい。Git たちがうのは `git status` は commit されてない working tree と staging area の状態を確認するためのものである一方、`jj status` は commit 済みの working copy の状態を確認するためのものであること。このコマンドで表示される情報は以下のとおり」

- その working copy でどのような変更が行われたかのサマリー

²¹ 「`jj status` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-status>

- その commit と親 commit の情報
- conflict が発生している場合はその情報

「プロジェクト作成時に作成された大量のファイルの名前が、頭に A をつけて並んでいますね。これってさっきの説明によればすでに commit されてるってことでいいんでしょうか？」

「そのとおり。ではその commit の履歴を見てみよう」

```
$ jj log
@ uwzmrwo kanaetti@skydome.co.jp 2026-03-20 17:47:11 ba953de6
| (no description set)
◆ zzzzzzzz root() 00000000
```

「commit を並べただけの git log とだいぶちがいますね」

「jj log²² は履歴がツリーグラフで表示されるうえ、個別の表記もコンパクトにまとまっているので、すぐ見やすくなってる」

「一番下の zzzzzzzz とか 00000000 という冗談みたいな ID が書かれてる履歴は何でしょう？」

「これは **Root Commit** といってすべての commit 履歴の基点となる、親を持たない特殊な『空の commit』なの。リポジトリの初期状態を表す仮想 commit であり、通常の commit とちがって編集や削除がまったくできない」

「だから変な ID が設定されてるんですね。ところで現在の commit はその上のノードが @ で表示されているところですよ。こっちは uwzmrwo と ba953de6 というまともそうな ID が振られています。それにしてもどうして ID が2つもあるんですか？」

「いい質問だね。じゃあ実験のために working copy を少しいじってみようか」

²² 「jj log - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-log>

```
$ echo 'Hello!' > hello.txt
$ jj log
@ uwzmwrwo kanaetti@skydome.co.jp 2026-03-20 17:48:07 3861e729 ←ここに注目
| (no description set)
◆ zzzzzzzz root() 00000000
```

「あっ！ 右側の ID が `ba953de6` から `3861e729` に変わった！」

「そう。Jujutsu の履歴は `commit` 単位ではなくそれを包括する単位である `change` が主体というのはさっきも少し話したけど、左の `uwzmwrwo` が **Change ID**、右の `3861e729` が **Commit ID** なのね。working copy が変更されて新しく `commit` が作られると `commit ID` は変化するっぽう、`change ID` は変わらない」

「不変の `change ID` と可変の `commit ID` ですか。変わらないのに `change ID` とはこれ如何に？」

「まあ無理問答みたいだね。`change` の概念を今の時点でちゃんと理解するのは難しいだろうけど、とりあえず今はひととおり体験することを優先させるね。

ところで、この `change` はよく見ると `(no description set)` とあるように **Description**、Git でいうところの `commit message` がない。次の作業に移る前に `jj describe`²³ でつけておこう」

```
$ jj describe -m "create project"
Working copy (@) now at: uwzmwrwo d100ade2 create project
Parent commit (@-)      : zzzzzzzz 00000000 (empty) (no description set)
```

「そしてこれは `git commit` でも同様だけど `-m` オプションを省略するとエディタが起動して、そこで本文を書くことが要求される。Jujutsu の `description` は気軽に書き換えられるものなので、エディタで慎重に書いたりせず `-m` を使ってコマンド一発で書くほうがおすすめだね」

²³ 「`jj describe` - CLI reference - Jujutsu docs」(エイリアス: `jj desc`)
<https://www.jj-vcs.dev/latest/cli-reference/#jj-describe>

2-2-3. Change の操作

「`pnpm run dev` を実行して開発サーバを起動し、ブラウザで `http://localhost:5173/` にアクセスすると、先ほど作成した React のテンプレートアプリの画面が表示される。これは `useState` API²⁴ を使った簡単なカウンターだね。ボタンを押すとカウントが1ずつ増加するという単純なもの」

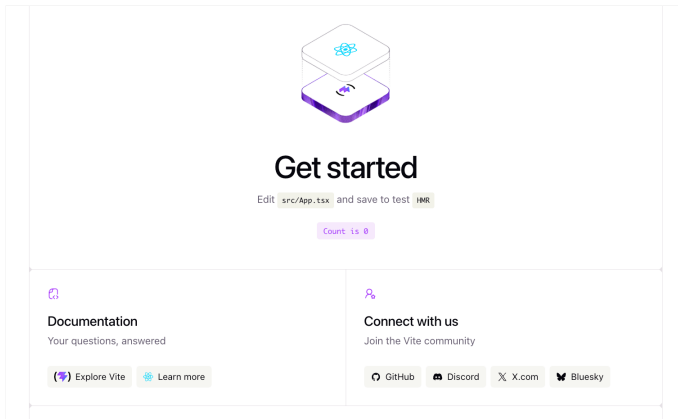


図 4: React テンプレートアプリの起動画面

「さてここで、このインクリメントの数値を2に変更したい。そのために新しく `change` を作って²⁵、そこで作業することにする」

```
$ jj new
Working copy (@) now at: qqwrqmns d6d64c92 (empty) (no description set)
Parent commit (@-)      : uwzmmrwo d100ade2 create project

$ vi src/App.tsx      # インクリメント値を 2 に変更する
$ jj diff
```

²⁴ <https://ja.react.dev/reference/react/useState>

²⁵ 「`jj new` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-new>

第2章 使ってみよう Jujutsu

```
> jj diff
Modified regular file src/App.tsx:
...
23 23:      </div>
24 24:      <button
25 25:          className="counter"
26 26:          onClick={() => setCount((count) => count + 12)}
27 27:      >
28 28:          Count is {count}
29 29:      </button>
...
```

図 5: `jj diff` の実行結果

「`jj diff`²⁶ って Git と diff の表現形式がちがいますね。行番号が表示されて、差分はインラインで赤と青に色分けして表示されてる。差分が多いときは、こっちのほうがコンパクトにまとまって見やすそう」

「これは `color-words` 形式²⁷ といって、Jujutsu が独自に実装しているもの。実は `git diff` にも `--color-words` オプションはあるんだけど別物で、Jujutsu のほうが行番号があったり、差分がよりピンポイントで表示されたりと高性能だね。なお、Git 同様の patch 形式で出力したい場合は `--git` オプションが使える」

「へー、そんな細かいところまで Git 互換が考慮されてるんですね」

「続けて、もう少し作業を重ねていこう」

```
$ jj desc -m "change increment value 1 -> 2"
Working copy (@) now at: qqwrqmns 0dbd6967 change increment value 1 -> 2
Parent commit (@-)      : uwzmwrwo d100ade2 create project

$ jj new -m "remove unnecessary file"
$ rm hello.txt
$ jj log --summary
@ on1kuqxy kanaetti@skydome.co.jp 2026-03-20 17:50:23 a68c193c
| remove unnecessary file
| D hello.txt
o qqwrqmns kanaetti@skydome.co.jp 2026-03-20 17:49:52 0dbd6967
```

²⁶ 「`jj diff` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-diff>

²⁷ 「Color-words diff options - Diff format - Settings - Jujutsu docs」
<https://docs.jj-vcs.dev/latest/config/#color-words-diff-options>

```

| change increment value 1 -> 2
| M src/App.tsx
o uwzmrwo kanaetti@skydome.co.jp 2026-03-20 17:48:52 d100ade2
| create project
| A .gitignore
| A README.md
| :
| A vite.config.ts
◆ zzzzzzzz root() 00000000

```

「あ、`jj log --summary` でどのファイルが変更されたかまで表示できるんですね。これなら `jj status` を使う機会はけっこう少なそう」

「いいところに気がついたね。Git だと常にどのファイルが index されてるかを気にする必要があるので、しょっちゅう `git status` を叩く必要がある。いっぽう Jujutsu では working copy が即 commit されるので、ユーザーの関心事は commit 間の差分だったり、履歴ツリー上のどこに今いるのかといったことになる。ゆえに Jujutsu では `jj log` がもっとも多用されるコマンドで、`jj` コマンド単体のデフォルトの挙動が `jj log` になっているほどなの」

「なるほど、納得です」

「次は過去の履歴を書き換えよう。Jujutsu では気軽に履歴を行ったり来たりできるのを体験してもらいたい」

```

$ jj edit qqwrqmns
$ jj log
o onlkuqxy kanaetti@skydome.co.jp 2026-03-20 17:50:23 a68c193c
| remove unnecessary file
@ qqwrqmns kanaetti@skydome.co.jp 2026-03-20 17:49:52 0dbd6967 ←ここに移動
| change increment value 1 -> 2
o uwzmrwo kanaetti@skydome.co.jp 2026-03-20 17:48:52 d100ade2
| create project
◆ zzzzzzzz root() 00000000

$ vi src/App.tsx      # インクリメント値を 5 に変更する
$ jj desc -m "change increment value 1 -> 5"

```

```
$ jj edit onlkuqxy
$ jj log
@ onlkuqxy kanaetti@skydome.co.jp 2026-03-20 17:51:48 4b0da050 ←ここに戻る
| remove unnecessary file
o qqwrqmns kanaetti@skydome.co.jp 2026-03-20 17:51:48 7d0ceb70
| change increment value 1 -> 5
o uwzmrwwo kanaetti@skydome.co.jp 2026-03-20 17:48:52 d100ade2
| create project
◆ zzzzzzzz root() 00000000
```

「`jj edit`²⁸ は working-copy commit (`@`) を履歴の任意のポイントに移動させるコマンド。ここで何をやったかがわかる？」

「えーっと、まずインクリメント値を変更した `change` に移動して値を 2 → 5 に書き換えた。さらにその description を `change increment value 1 -> 5` に変更した上で、最新の `change` に戻ってきたと。合ってるでしょうか？」

「はい、ご名答。Jujutsu では履歴の書き換えがめっちゃ簡単でしょ？」

「……いや、シンプルすぎてまだ理解が追いついてません。過去の履歴を書き換えたら、それを現在に反映させるには `rebase` が必要なのはですね。いったいどうなってるんですか？」

「これが Jujutsu のすごいところでね。commit を変更すると、すべての子孫が新しく変更された commit の上に自動的に `rebase` されるようになってるの。もし途中で `conflict` が起きても、それが発生した履歴に `conflict` マークがつくだけで自動 `rebase` 自体は成功する。conflict を解決するには該当の commit を修正すれば、それも自動的に子孫に伝播する」

「驚きました。まさにタイムマシンで過去と現在を自由に行き来できる感覚ですね」

「今度は、任意の履歴を破棄してみよう²⁹」

²⁸ 「`jj edit` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-edit>

²⁹ `jj abandon` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-abandon>

第3章 実践 Jujutsu × AI ワークフロー

3-1. AI エージェントを Jujutsu と協調させる

3-1-1. AI エージェントに Jujutsu を使わせる

「私は最近 Git の操作も AI に任せるようになって、『じゃあここまでのところを commit して push して』みたいによく頼んでるんですね。Jujutsu の操作を AI に任せることってできるんですか？」

「もちろんできるよ。Jujutsu なら undo があるし、AI が一気に行った作業の途中の時点に巻き戻すなんてことも簡単にできるから、むしろ Git よりも思い切って操作を任せられる」

「へーえ。ちなみに雪菜さんはずっと AI 支援開発と併せて Jujutsu を使ってるんですね。普通に AI エージェントに『Jujutsu を使って』と指示するだけではうまくいかないように思えるんですが、どんな工夫をしてるんですか？ そのあたりの知見はめっちゃ貴重だと思うので、ぜひ共有してくれると嬉しいです！」

「わかった。ただ AI エージェントごとに機能や特性がけっこうちがうし、目まぐるしく新機能が追加されている最中なので、ベストプラクティスはなかなか定まらない。コーディングエージェントの中では Claude Code が最大シェアで私もヘビーユーザーなので、その 2026 年 3 月時点での最適と思われる方法を中心に説明しよう。あと Codex CLI についてもその都度、補足的に触れておこうか」

「お願いします。私も日常的に使ってるのはほぼその 2 つなので」

「じゃあまずは、AI エージェントに VCS として Git ではなく Jujutsu を使ってもらうための指針を示す方法から説明しようか。Claude Code では **Rules** 中心で運用するのがいいと思う⁴⁶」

「`CLAUDE.md` じゃなく？」

「それも使うよ。ただしそっちには短く、最優先で守ってほしい事項だけを書い

⁴⁶ 「Claude があなたのプロジェクトを記憶する方法 - Claude Code Docs」
<https://code.claude.com/docs/ja/memory>

ておく。そして本編を rules として記述するの」

「複雑ですね。どっちかに統一できないんですか？」

「`CLAUDE.md` の限界は、本質的にプロジェクトのコンテキスト情報を伝えるためのものであること。そこに『このプロジェクトでは Jujutsu を使っています』と書いてもただの情報として処理され、Claude Code がタスクに没頭するうちに他の情報に埋もれていき、重要度が下がってしまう。特に `CLAUDE.md` の内容が長いほどそうなりがち」

「たしかに、ずっと作業を続けると `CLAUDE.md` や `AGENTS.md` の内容が無視されることってめずらしくないですよ」

「AI コーディングエージェントは学習データのほとんどが Git を使ったものだから、そのデフォルトの行動として Git のワークフローが深く組み込まれてしまってるのね。変更を確認したければ `git diff`、commit したければ `git commit`、branch を作りたければ `git checkout -b`。これらは単なる知識じゃなく、条件反射的な行動パターンになってる。これを矯正するのは rules のほうが適してるの」「それはなぜでしょう？」

「rules はすべてのインタラクションに常時適用される行動制約として設計されてるからだね。セッション中のすべてのやり取りに注入されるので、Claude Code がどんな文脈で VCS コマンドを使おうとしても、『`git` ではなく `jj` を使え』という指示が常時介入してくる」

「なるほど、よくわかりました。ところで skills はどうですか？ `skills.sh`⁴⁷ で『`jujutsu`』を検索すると、けっこういろんな skills がヒットします。これだけあるってことは、それなりに使われてるように思えるんですが」

「たぶんうまくいかないんじゃないかな。skills は AI エージェントが『これは関連がある』と判断したときに読みに行くリファレンスであって、常時適用される行動制約じゃないからね。多くの場合は明示的に指定しないと使ってくれない。VCS 操作はコーディング作業の途中で暗黙的に発生するものだから、skills の参照がトリガーされないまま `git` コマンドが実行される可能性が高い」

⁴⁷ Vercel が 2026 年 1 月に公開した、AI エージェント用の skills を検索・管理できるオープンなディレクトリ兼ランキングサイト。
<https://skills.sh/>

3-1. AI エージェントを Jujutsu と協調させる

「うーん、それなら rules に加えて skills も導入して併用するのはどうでしょう？」
「それもやめたほうがいいだろうね。指示が競合するリスクがあって、両方がコンテキストに入るとエージェントを混乱させてしまう。また用語の不整合も気になる。たとえば同じ『commit』でも Git と Jujutsu ではニュアンスがちがうというのは説明したけど、これらの skills はそのあたりのことをほとんど考慮してないで、それも混乱の元になる」

「なるほど」

「あと駄目押しに permissions 設定も使っておきたい。これについては別途、あとで説明することにしてよう。」

さて、その rules のファイルには何を書いたらいいか。私はおおむね次のようなことを記述してる」

1. Git コマンドの使用禁止
2. Git と Jujutsu での用語およびメンタルモデルのちがいを
3. diff 出力のコマンド実行には `--git` をつけること
4. ファイル変更後の確認以外では、参照系コマンドに `--ignore-working-copy` をつけること
5. conflict マーカーの読み方
6. Jujutsu でのユースケース別ワークフロー

「1 は説明するまでもないよね。2 については、AI が意図通りに動作してくれるような表現に落とし込むのにすごく苦労したよ。最終的に、説明の正確さを追求せずに用語の対応表を作ったうえで、やっちゃだめなことを直接書く形にした。だから『commit = change』とか『HEAD = working copy』とか技術的には正しくないことも書いてある。また Jujutsu の文脈で commit という言葉を使うことを極力避けて、revision という表現を使うようにしてる」

「AI に Jujutsu を正しく理解させることを半ばあきらめて、とりあえず動くことを優先させたんですね。実際に使い込んだ人じゃないととどろけない境地だ」

「AI エージェントに大きく変わることを求めず、Git 脳の認知的負荷を軽減してあげることを心がけた。diff 出力を Git 形式にすることもそうだね。これは Jujutsu の設定でも同じことができるんだけど、それをやると今度は人間が読みにくく

なるから、コマンド実行時に `--git` オプションをつけてもらうことにしてる」

「4の `--ignore-working-copy` というのは？」

「`working copy` と履歴に差分がある状態で `jj` コマンドを実行すると、通常はスナップショットがとられるよね。その挙動を無効にするのがこのオプション。

AI が作業を終えるのを待っているとき、自分は別のペインで他の作業をすることがよくあるよね。もし AI による `jj` コマンドの実行と自分によるそのタイミングが重なった場合、一方のコマンドの実行が終わらないうちにもう一方がスナップショットをとって履歴を進めてしまうことがある。するとそのプロセスから見て `working-copy commit` が履歴の示す `working-copy commit` より古くなってしまい、その不整合からエラーが発生する。そういった事態をできるだけ防ぐために指定してるの」

「ふーむ、そんな配慮も必要なんですね」

「それから、Codex CLI でのやり方についても説明しておこうか。Codex には 2026 年 3 月時点で Claude Code のような `rules` の機能がない。同じ名前でも `rules` という機能があるけど、こっちはサンドボックス外で実行できるコマンドを管理するためのものであって、自然言語で AI エージェントに行動規範を与えるものじゃない」

「困りましたね。じゃあどうするんですか？」

「`AGENTS.md` と `skills` の併用で乗り切るのが次善策かな。ただしさっきも話したように、`skills` はよほど関連があると AI が判断しないかぎり使ってくれない。だからこちらの場合は `AGENTS.md` の記述が多めになる」

「うーむ、さじ加減が難しいんですね。こんなに AI に合わせてアプローチの方法を変えないといけないとは」

「ここまでの話を総合すると、プロジェクトごとの AI エージェント用の設定ファイルはこんな感じで置くことになる⁴⁸」

⁴⁸ <https://github.com/kleimiary/Juju-chu-ja/tree/main/config/ch3>

3-1. AI エージェントを Jujutsu と協調させる

リスト 1: エージェント用設定ファイルの配置

```
my-project/
  CLAUDE.md          ← Claude Code に最優先で守らせたい事項を記述
  AGENTS.md          ← Codex にぜひとも守らせたい事項を記述
  .claude/
    rules/
      jujutsu-rules.md ← 該当の rules ファイル
  .codex/
    skills/
      jujutsu/
        SKILL.md      ← 詳細を skill 化したもの
```

リスト 2: jujutsu-rules.md の冒頭部分

```
# Jujutsu (jj) Rules for AI Agents

このプロジェクトではバージョン管理に **Jujutsu (jj)** を使用する。AI エージェントは以下のルールを厳守し、**`git` コマンドは原則使用禁止**とする（`jj git` サブコマンドおよび `gh` CLI を除く）。

---

## AI エージェント向け重要注意

### 用語のマッピング

Jujutsu と Git では用語の意味が異なる。AI は以下の用語の使い分けを徹底すること。



| Git 用語                 | Jujutsu 用語   |
|------------------------|--------------|
| -----                  | -----        |
| commit (名詞として)         | change       |
| branch                 | bookmark     |
| staging                | (この概念は存在しない) |
| unstaged / uncommitted | (この概念は存在しない) |


```

```
|  
| HEAD | `@` (working copy)  
|  
| stash | (この概念は存在しない。`jj new` で代替)  
|  
| `git add` | (不要。自動スナップショット)  
|  
| `git commit --amend` | (不要。`@` への変更は自動的に反映される)  
|
```

Jujutsu の Git との根本的な違い

1. ****「未保存の変更」は存在しない****: ファイルを保存した瞬間に現在の change に自動的に含まれる。「この変更を含めますか？」という確認は不要である。
2. ****change と revision****:
 - ****change****: 作業単位。一意で不変の ****change ID**** を持ち、内容は可変 (mutable)。
 - ****revision****: change のスナップショット。編集のたびに新しい revision が作られるが、change ID は不変。
3. ****自動 rebase****: change の親を変更すると、子孫の change が自動的に rebase される。手動で rebase チェーンを管理する必要はない。
4. ****first-class conflict****: conflict が発生しても操作は中断されず、conflict を含む change として記録される。後から解決可能。
5. ****operation log****: すべての操作が記録されており、`jj undo` / `jj op restore` で任意の時点に戻れる。失敗を恐れず操作してよい。
- ⋮

「既存のリポジトリは Git で管理してるはずだから、ユーザー設定ではなく Jujutsu を使っているリポジトリごとに設定を置くのがいいだろうね」
「いいですね！ これそのまま、いただいちゃいます！」

⁴⁹ 「権限を設定する - Claude Code Docs」
<https://code.claude.com/docs/ja/permissions>

3-1-2. jj コマンドの Permissions 設定

「Claude Code の **Permissions 設定**⁴⁹は、任意のコマンドの実行を **allow**（自動許可） / **ask**（都度確認） / **deny**（拒否）の3種類のルールに従わせることができるというもの。先の rules には『Git コマンドは使用禁止』といったことを書いたけど、この permissions 設定なら確実に禁止できる。また他の jj コマンドそれぞれについても、いちいち対話で許可しなくても実行していいものを列挙しておけば、開発がスムーズに進められるよね」

「ああ、あの面倒くさいから毎回『Always allow』を選択してたら、`settings.local.json` にどんどん追加されていくやつですよ」

「そうなんだけど、設定はちゃんと定期的に管理したほうがいいね。私もそうやって `settings.local.json` から育てた `settings.json` があるので、その中から Jujutsu に関係するものだけを抽出して紹介しよう」

リスト 3: settings.json の permissions 設定

```
{
  "permissions": {
    "allow": [
      "Bash(jj status:*)",
      "Bash(jj log:*)",
      "Bash(jj diff:*)",
      "Bash(jj show:*)",
      "Bash(jj file list:*)",
      "Bash(jj root:*)",
      "Bash(jj describe:*)",
      "Bash(jj new:*)",
      "Bash(jj edit:*)",
      "Bash(jj restore:*)",
      "Bash(jj squash:*)",
      "Bash(jj rebase:*)",
      "Bash(jj abandon:*)",
      "Bash(jj evolog:*)",
      "Bash(jj op log:*)",
      "Bash(jj operation log:*)",
      "Bash(jj undo:*)",
    ]
  }
}
```

```
"Bash(jj bookmark list:*)",
"Bash(jj bookmark create:*)",
"Bash(jj bookmark move:*)",
"Bash(jj bookmark advance:*)",
"Bash(jj bookmark set:*)",
"Bash(jj bookmark rename:*)",
"Bash(jj git fetch:*)",
"Bash(jj config get:*)",
"Bash(jj config list:*)"
],
"deny": [
  "Bash(git:*)",
  "Bash(jj split:*)",
  "Bash(jj resolve:*)",
  "Bash(jj diffedit:*)",
  "Bash(jj arrange:*)",
],
"ask": [
  "Bash(jj bookmark delete:*)",
  "Bash(jj bookmark forget:*)",
  "Bash(jj bookmark track:*)",
  "Bash(jj bookmark untrack:*)",
  "Bash(jj git push:*)"
]
}
}
```

「うわー、きれいに整理されてますねえ。こういうところに性格が出るなあ」

「ふふ、ありがと。じゃあこの内容について、`deny` と `ask` だけ軽く説明しておこうか。

まず Git コマンドは全面禁止。あと差分エディタやマージツールなどが起動してインタラクティブに編集するようなコマンドも、AI が操作できないので禁止してる。bookmark の破壊的操作と push はリモート絡みなので、確認しながらやりたいから `ask` にしてる」

「ふむふむ」

「そして Codex CLI の場合にも触れておく。permissions と一見似たような機能と

3-1. AI エージェントを Jujutsu と協調させる

して **Rules**⁵⁰ があるんだけど、これは少し意味がちがう。そもそも Codex はサンドボックス環境で動くしくみになっていて、その中で完結するコマンドであればユーザーの承認を必要とせず実行できるのね」

「あれ？ でも Codex を使ってるとき、Git コマンドの許可をいちいち求められましたよ？」

「うん、それは `.git/` ディレクトリがシステムで読み取り専用として保護されているから。あとリモートとやり取りする際もサンドボックスの外に出るので、やっぱり許可が必要になる。というわけで、Jujutsu のコマンドもけっきょく、Claude Code の Permissions に相当する設定を rules に書いておく必要があるの。

ただし rules の設定はサンドボックスの外に出ようとしたときに適用されるものなので、最初から禁止したいコマンドは別途、`AGENTS.md` に書いておいたほうが無難だね」

「これも複合戦略ですか……。けっこう面倒ですね」

「コピーさせてあげるんだから、文句言わないの。それで今回、追加・変更するファイルは以下のとおりね⁵¹」

リスト 4: エージェント用設定ファイルの追加・変更

```
my-project/
  CLAUDE.md
  AGENTS.md          ← 禁止コマンドを記述
  .claude/
    settings.json    ← permissions 設定を記述
    rules/
      jujutsu-rules.md
  .codex/
    rules/
      jujutsu.rules  ← 実行ルール設定を記述
    skills/
      jujutsu/
        SKILL.md
```

⁵⁰ 「Rules – Codex | OpenAI Developers」
<https://developers.openai.com/codex/rules>

⁵¹ 同じく <https://github.com/kleimiary/Juju-chu-ja/tree/main/config/ch3>

第 4 章 Jujutsu の高度な術式

4-1. ターゲットを指定する冴えたやりかた

4-1-1. Revset で Revision をスマートに指定する

「ここまであまり説明することなく、ほとんどの場面でコマンドでの revision 指定に ID を使ってきた。でも実は Jujutsu には **Revset** (リブセット)⁷² という強力な式言語が用意されていて、それを使えば各種コマンドでもっと多彩なことが便利にできるようになるの」

「へー。そういえば Git でも `HEAD~3` とか `main..HEAD` みたいな指定ができましたよね」

「うん。ただ Git のはあくまで記法であって表現できることは限られてる。Jujutsu の revset は Mercurial 譲り⁷³の関数型言語で、ものすごく表現力が高い。ここではよく使いそうなものに絞って説明しようと思う」

シンボル

「まずは、もう何度も使ってるけど `@` というシンボル。これはその workspace における working-copy commit、つまり現在の change の最新 revision を指す。また `@` の記号自体は他の用途にも使われていて、`todo-app-blue@` のように末尾につけると workspace、`main@origin` のように記述するとリモートの bookmark の指定になる」

「へー、`@` もその revset のひとつだったんですね」

「あと change ID や revision ID もいちおうシンボルに分類されることになってる。ところで `jj log` のグラフに表示されてる change ID や revision ID をあらためてよく見てみて。何か気付くことはない？」

⁷² 「Revset language - Jujutsu docs」 <https://www.jj-vcs.dev/latest/revsets/>

⁷³ 「Help: revsets」 <https://repo.mercurial-scm.org/hg/help/revsets>

```
@ ypxzsvos kanaetti@skydome.co.jp 2026-03-26 19:58:16 default@ ed2b89ac
| chore(merge): integrate filter feature into default
| o moqzsrnw kanaetti@skydome.co.jp 2026-03-26 18:00:50 todo-app-blue@ a1b80d97
| | fix: suppress noStaticElementInteractions error
| o mvqzlnpq kanaetti@skydome.co.jp 2026-03-26 17:43:11 fdb8f7a2
| | feat: add filtering for completed and incomplete todos
```

図 12: ログの ID の先頭が太字と色で強調されている

「.....そういえば先頭の数文字が太字になってたり色が変わってたりしますよね。なんですかこれ？」

「実は ID をフルで記述しなくても、この強調されてる数文字を入力するだけで revision が特定されるの。Unique Prefix っていうんだけど」

「ええー、もっと早く教えてくださいよ。コピペするためにキーボードから手を離さなくて済むじゃないですか」

「最初は苦勞したほうがありがたみがあるでしょ。ちなみに operation log ではこんなふうに強調表示されないんだけど、実は同様に省略できる。ただ何文字めまでなら特定されるか事前にわからないので、無難に 3~4 文字くらいは入力しておいたほうがよさそうね」

演算子

「いわゆる演算子もサポートされてる。主なものを挙げておこう」

- `x-` — `x` の親。 `x--` だとさらにその親を指す
- `x+` — `x` の子。 `x++` だとさらにその子を指す
- `x::y` — `x` の子孫であり、かつ `y` の祖先でもある revision。 `x::` は `x` の子孫すべて、 `::x` は `x` の祖先すべてを指す
- `x..y` — 上記とほぼ同じだが、 `x` は含まない。 `..x` は root commit を除いた `x` の祖先を指す
- `~x` — `x` 以外のすべて
- `x | y` — `x` および `y`
- `x & y` — `x` と `y` が指すものの双方に含まれる revision
- `x ~ y` — `x` が指すものから `y` を除いた revision

「以前、省略されないすべての revision を見るために `jj log -r ::` を実行したことがあったよね。この説明のとおり、起点と終点を省略された `::` は root commit から直近の末裔までを指すから、すべての revision が該当することになるわけね」

「なるほど、そうだったんですね。ところで `x` のひ孫だと `x+++` と書かなきゃいけないんでしょうか？ Git の `^5` みたいに数字で指定できない？」

「それは残念ながらできない。長い世代を隔てた指定には、次に説明する関数を使ったほうがいいだろうね」

関数

- `mine()` — 自分が作成者の revision
- `conflicts()` — conflict が発生している revision
- `mutable()`, `immutable()` — mutable および immutable な revision
- `parents(x, [depth])`, `children(x, [depth])` — `x` の親および子の revision。
`parents(x, 2)` は `x--` に等しい
- `ancestors(x, [depth])`, `descendants(x, [depth])` — `x` の祖先および子孫の revision。
`ancestors(x, 3)` は `x--::x` に等しい
- `description(pattern)` — `pattern` にマッチする description を持つ revision
- `bookmarks([pattern])` — ローカルの bookmark が指す revision。引数がない場合はすべて、`pattern` が指定された場合はそれにマッチするものが対象
- `tags([pattern])` — タグが指す revision。引数がない場合はすべて、`pattern` が指定された場合はそれにマッチするものが対象

「`revset` の関数は 50 個くらいあるんだけど、めばしいものはこのあたりかな。さっき質問された `x` のひ孫は `children(x, 3)` になるね。気をつけないといけないのは、`parents()` と `children()` の `depth` は自分を含めないで数える一方、`ancestors()` と `descendants()` では自分を含むこと。だからひ孫に至るまでの子孫を指定したい場合は `descendants(x, 4)` と `depth` に 1 を足す必要がある」

「うーむ、ややこしいですね」

文字列パターン

- `glob:"string"` — glob パターン（※デフォルト）
- `regex:"string"` — 正規表現
- `exact:"string"` — 完全一致
- `substring:"string"` — 部分一致

「関数に渡せる文字列パターンはこの 4 種類。何も指定しなければ `ls` コマンドなどで使われてる glob パターンが適用される。たとえば `feat:` から始まる `description` を持つ `revision` のログを見たい場合は `jj log -r 'description(regex:"^feat:")'` を実行する」

「なるほど、そうやって使うんですね。ところで `revset` を指定するのにクォーテーションで囲っている場合とそうでない場合があるのはどういうことですか？」

「ID とシンボル、およびそれによる範囲指定のみの場合はクォーテーションで囲う必要はないよ。それ以外は式として評価させるためにクォーテーションがいる。なお一番外のクォーテーションは必ず `'` のシングルクォートを使うようにすること。シンボルや演算子がシェルの特許文字を含んでるため、意図しない挙動になることがあるからね」

「わかりました」

4-1-2. Fileset でファイルをスマートに指定する

「これも疑問に思ってたんですけど、`revision` を指定するのに `-r` オプションが必要なコマンドと直接渡せるコマンドがあるのはどういうことですか？ `jj log` は `-r` がいるのに、`jj edit` はいらなかったですね？」

「いい質問だね。基本的に『`revision` そのものを操作の主な対象としているコマンド』は、`revision` を位置引数として直接受け取るから `-r` は必要ない。`jj edit` や `jj abandon` がそうだね。

いっぽう『ファイルパスを位置引数に取るコマンド』は対象の `revision` を特定

第 5 章 Jujutsu 問題解決ガイド

5-1. FAQ

5-1-1. Git との比較

🗨️ Git にできて Jujutsu にできないことは？

「Git から移行するにあたって聞いておきたいんですけど、Git ではできるのに Jujutsu ではできないことって何があるんでしょうか？」

「すでに Git hooks に相当する機能が Jujutsu には存在しないということは話したよね¹⁰⁰。それ以外で目ぼしいものだと、次のようなものが挙げられる」

- タグの push

..... Jujutsu には `jj tag` コマンドがあり、タグの作成や削除、一覧表示はできるものの、リモートへの `push` には対応していない

- サブモジュールの管理

..... 任意の Git リポジトリを別の Git リポジトリのサブディレクトリとして扱うこと（＝サブモジュール）が、Jujutsu では未対応。ロードマップにはあるので¹⁰¹、将来的にはサポートされそう

- LFS (Large File Storage) の取り扱い

..... Git LFS は大容量ファイルを Git で効率的に管理する拡張機能。GitHub では 100MB 以上のファイルをバージョン管理する場合に LFS の使用が必要だが、Jujutsu では未サポート

- `.gitattributes`

..... Git はリポジトリ内のファイルやディレクトリ単位で、改行コード

¹⁰⁰ 「4-3. Git Hooks の代替戦術」を参照のこと。

¹⁰¹ 「Submodule support - Development roadmap - Jujutsu docs」
<https://docs.jj-vcs.dev/latest/roadmap/#submodule-support>

の統一やバイナリ／テキストの識別、マージ戦略などのカスタム属性を `.gitattributes` ファイルで設定できるが、Jujutsu では未サポート

• Partial Clone

..... `partial clone` はオブジェクトの種類でフィルタリングして、不要なデータを省く `clone`。巨大リポジトリを扱うときに `clone` のコストを減らすためのしくみ。同等のオプションは `jj git clone` にはない

「知らなかった機能も多いですけど、タグの `push` ができないんですか」

「まあコマンドも `git push origin <tag>` と単純だし、ちょっと気持ち悪いけどそこだけ Git コマンドを使うということで」

🤖 merge コマンドはないの？

「Jujutsu って Git みたいに専用の `merge` コマンドがないですよね。どうしてでしょうか？」

「`revision` を `merge` したい場合、Jujutsu では `jj new` で複数の親を持つ新しい **change** を作成するという手法をとるようになってるの。すでに一度やってるはず¹⁰²。『ひとつの親から始める』のも『複数の親から始める (merge)』のも、新しい作業の起点を指定するという意味で同じであり、だから同じ操作で一貫して扱うべきというのが Jujutsu の思想なのね」

「`rebase` とちがって `merge` の場合は合流点の新しい `change` が作られるので、`jj new` で行えばいいということですか」

「そう。加えて Jujutsu では `conflict` が起きても操作がブロックされず、それを単なる `revision` の状態として保持するからね。解消は後回しでもいいし、何度でもやり直せる。この設計によって `merge` を Git のように『失敗する可能性のある、慎重に行うべき儀式』として扱う必要がなく、通常のフローに統合できる。

また実質的に `new` と変わらない専用の `merge` コマンドがあると、ユーザーが Git からのイメージを引きずって Jujutsu が想定するワークフローに乗ってくれ

¹⁰² 「3-2-3. workspace を使った並行開発」を参照のこと。

ないという懸念もあると思う」

「ふむふむ。作ろうと思えば作れるけど、思想・信条からあえて作らないと」

「実は `jj merge` コマンドは昔は存在してたんだけど、2024 年 2 月リリースの 0.14.0 で非推奨になったという経緯がある¹⁰³。その理由は、コミュニティ内でさっき説明したような議論があったから。個人的な見解だけど、この変更は結果的に Jujutsu を Git の代替品ではなく、それを超える新しい VCS としての自己を確立するための転機になったんじゃないかな。

`checkout` も `merge` もなく、すべてが `new` から始まる。`jj new` は Jujutsu の設計思想が凝縮されたコマンドだと言えるだろうね」

🤔 pull コマンドはないの？

「Git だとそのブランチの最新の変更を取り入れるには `git pull --rebase` 一発で済むのに、Jujutsu だと `jj git fetch` と `jj rebase -o main` と 2 つのコマンドを実行する必要がありますよね。pull コマンドがあったほうが便利なのに、なぜないんでしょうか？」

「`git pull` は、`git fetch` と `git merge` / `git rebase` を一括実行してくれるコマンドだよ。 `--rebase` オプションの有無で後半が `merge` なのか `rebase` なのかが変わるといってかなり変則的な仕様だけど、まずそのようなコマンド体系が Jujutsu にはなじまない。 `revset` がほぼすべてのコマンドに統一的に利用できるように、オプション指定にも一貫したポリシーがあるの。コマンドオプションはあくまで、どの対象をどのように操作するかを指定するものであり、それによって動作の種類が根本的に変わるものであってはならない。

また Jujutsu では操作の明示性も大事にされてる。 `fetch` と `merge` / `rebase` は本質的にレイヤーの異なる操作であり、ツリーグラフに与える変化が別物だよ。それが `operation log` で統合されてしまうのもユーザーにとって不利益になる」

「あー、たしかに。 `undo` するにしても、別々にやりたいですね」

¹⁰³ 「0.14.0 - 2024-02-07 - Changelog - Jujutsu docs」
<https://www.jj-vcs.dev/latest/changelog/#0140-2024-02-07>

「Jujutsu では、ユーザーがグラフ構造や自分が行う操作を明確に把握することを好むの。だから『ユーザーがわかってなくても、まとめてうまいことやってくれる』おまじないのようなコマンドは、これからも作られないと思う」

☹️ Git の cherry-pick 相当のことがしたい

「別の branch にある特定の change をピンポイントで working copy に持ってきたい、つまり `git cherry-pick` のようなことを Jujutsu でもやりたい場合、どんなコマンドを実行すればいいですか？」

「その場合は、既存の change と同じ内容の新しい change を作る `jj duplicate`¹⁰⁴ というコマンドを使う。具体的には複製した change を working copy の直下に挿入して、working copy をその上に rebase する必要があるので、実行するのは以下のようなコマンドになる」

```
$ jj duplicate <source> -B @
```

「`-B` に相当するロングネームオプションは `--insert-before` ね。rebase は自動的に行われるので、あらためて実行する必要はないよ」

5-1-2. ニッチな操作・設定について

☹️ @ を移動させずに、任意の時点でのファイルの内容を確認できる？

「任意の revision での変更ファイルの差分を見るのは `jj diff -r <revset> <fileset>` でできますよね。そうではなく、その時点でのファイル全体の内容を確認したい場合はどうしたらいいでしょうか？

`jj edit` で working copy をそこまで移動させればできるのは知っていますが、そ

¹⁰⁴ 「`jj duplicate` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-duplicate>

こも伸ばさず『ジュ→ジュ↑ツ→』と言ってるように聞こえる。ただこれを日本語の日常会話でそのまま発音すると違和感が大きいので、やっぱり『ジュ↑ジュ→ツ↓（※呪術と同じ）』の発音でいいと思う」
「なるほど。じゃあ私も今まで通り『ジュジュツ』でいきます」

5-2. トラブルシューティング

😓 ただの push が知らないうちに force push になっている

「Jujutsu を使って自分で色々と試してたんですけど、普通に push したけなのに GitHub 上からは force push になっているのを見つけました。なぜこういうことが起きるのでしょうか？」

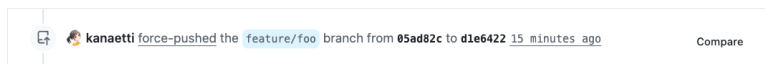


図 17: force push になった履歴

「ああそれね。そもそも Jujutsu の push は外からそう見えないだけで、内部的には常に `git push --force` に相当する操作を行ってるの。というのも Jujutsu は以下のような独自の安全策を用意してて、それによって履歴を守ってるので force push をタブー視してないのよ」

- リモートへの push は、自分が最後に fetch した状態からリモートが変わってない場合のみ成功する
(`git push --force-with-lease` 相当がデフォルトで有効)
- immutable commit はローカルでも変更不可

「immutable commit って、ログ上でノードが ◆ で表示されるやつでしたよね¹¹³」
「そう。あれはデフォルトでは `trunk()` | `tags()` | `untracked_remote_bookmarks()`

¹¹³ 「2-3-1. Revision Log (`jj log`)」を参照のこと。

という revset にマッチした revision が該当する¹¹⁴。つまり、`main@origin` の revision とタグ付きの revision、自分が track していない bookmark の revision は通常、書き換えができないようになってるのね」

「ふむふむ、それで？」

「しかしこれに main 以外の bookmark における push 済み revision は該当しない。だから Jujutsu からは自由に編集できるんだけど、それを Git のリモートリポジトリに push すると結果として force push として認識されてしまうわけ。でもこれは Jujutsu としては正常な動作。それは歴史を汚しているのではなく、『ひとつの change (論理的な作業単位) を最新の版に更新した結果』を反映させているに過ぎないの」

「なるほど、理由はわかりました。でもその branch を他のメンバーと共有してるときは、不都合が起きませんか？ force push が発生してるんだから、それを他の人が pull しようとするとう diverged 状態が発生しますよね」

「それはおっしゃるとおり。Jujutsu では、branch を複数メンバーで共有する運用はデフォルトでは想定されてないと言える。ただその場合も、設定をカスタマイズすることで対応可能。

feature branch (bookmark) の命名規則が `feature/` を頭に付けることになっている前提で、その push 済み bookmark の branch を immutable commit に含める設定は以下のようなになる」

```
[revset-aliases]
'immutable_heads()' = 'builtin_immutable_heads() | remote_bookmarks("feature/*", "origin")'
```

「これをユーザー用もしくはリポジトリ用の設定に追加しておけば、複数人で branch を共有する運用での diverged 事故は防げるはず」

¹¹⁴ 「Set of immutable commits - Settings - Jujutsu docs」
<https://docs.jj-vcs.dev/latest/config/#set-of-immutable-commits>

😓 『Error: The working copy is stale』という謎のエラーが出る

「なんかこんなエラーが出ちゃったんですけど、何が起こったんでしょう？ どうしたらいいですか？」

```
Error: The working copy is stale (not updated since operation 559e2c79f4ef).  
Hint: Run `jj workspace update-stale` to update it.  
See https://docs.jj-vcs.dev/latest/working-copy/#stale-working-copy for more  
information.
```

「ああ、working copy が stale 状態になっちゃったのね¹¹⁵。stale っていうのは『陳腐化した』って意味。つまり Jujutsu が『手元のファイルが私の知ってる最新の履歴と一致していないんだけど、このまま作業を続けてだいじょうぶ？』と尋ねてるの」「なんでそんなことが起きたんでしょう？」

「実はこの現象については、すでに何度か触れてるよ¹¹⁶。自分の `jj` コマンドの実行が、AI エージェントや UI ツール、別の workspace などによる `jj` コマンドの実行と微妙にタイミングが被ると、そちらが先にスナップショットをとって履歴を進めてしまい、自分がいるプロセスから見てる working-copy commit が履歴の示す working-copy commit より古くなるという事態が発生する。これが working copy の stale エラーね」

「そうでした。できるだけそのタイミングが被らないようにって注意してくれましたね。それでこれ、どうやって直すんですか？」

「落ち着いてエラーメッセージを見てみて。ヒントが書いてあるでしょう？ それをそのとおりに実行すればいい」

```
$ jj workspace update-stale
```

¹¹⁵ 「Stale working copy - Working copy - Jujutsu docs」
<https://docs.jj-vcs.dev/latest/working-copy/#stale-working-copy>

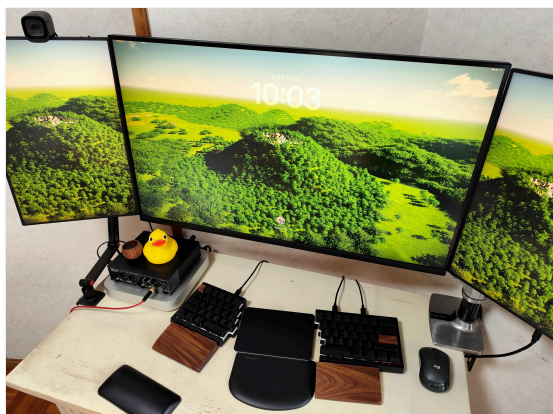
¹¹⁶ 「3-1-1. AI エージェントに Jujutsu を使わせる」および「4-4-2. JJ View」を参照のこと。

著者紹介

大岡由佳（おおおか ゆか）

国内の IT 企業数社での PHP や Ruby による開発およびプロダクトマネージャを経て、2016 年からフリーランスに転身。当時まだ日本ではマイナーな技術だった React に注目し、React 専門のフリーランスエンジニアとして複数の現場を渡り歩く。その経験を元に上梓した「りあクト！」シリーズが評判を呼び、累計部数 3 万部を超えるヒット作となる。

X アカウントは @oukayuka。ブログは <https://klemiwary.com>（くるみ割り書房公式サイト）。



著者のデスクトップ環境

イラスト：黒木めぐみ（くろき めぐみ）

漫画家、イラストレーター。

「りあクト!」「じゅじゅちゅ!」シリーズの表紙イラストを一貫して担当。

じゅじゅちゅ！ jj new で始める Jujutsu x AI ワークフロー

2026 年 4 月 10 日 第 1 刷発行

著 者	大岡 由佳
連 絡 先	oukayuka@gmail.com
発 行 元	くるみ割り書房 https://klemiwary.com

本書の一部、または全部を無断で複写、複製、転載、データ配信することを禁じます。

© 2026 Yuka Ooka / Klemiwary Books